Get Started

The best way to learn how to use Amazon SageMaker is to create, train, and deploy a simple machine learning model. To do this, you need the following:

- A dataset. You use the MNIST (Modified National Institute of Standards and Technology database) dataset of images of handwritten, single digit numbers. This dataset provides a training set of 50,000 example images of handwritten single-digit numbers, a validation set of 10,000 images, and a test dataset of 10,000 images. You provide this dataset to the algorithm for model training. For more information about the MNIST dataset, see MNIST Dataset.
- An algorithm. You use the XGBoost algorithm provided by Amazon SageMaker to train the model using the MNIST dataset. During model training, the algorithm assigns example data of handwritten numbers into 10 clusters: one for each number, 0 through 9. For more information about the algorithm, see XGBoost Algorithm (p. 255).

You also need a few resources for storing your data and running the code in this exercise:

- An Amazon Simple Storage Service (Amazon S3) bucket to store the training data and the model artifacts that Amazon SageMaker creates when it trains the model.
- An Amazon SageMaker notebook instance to prepare and process data and to train and deploy a machine learning model.
- A Jupyter notebook to use with the notebook instance to prepare your training data and train and deploy the model.

In this exercise, you learn how to create all of the resources that you need to create, train, and deploy a model.

> **Important**
> For model training, deployment, and validation, you can use either of the following:
>
> - The high-level Amazon SageMaker Python SDK
> - The AWS SDK for Python (Boto 3)
>
> The Amazon SageMaker Python SDK abstracts several implementation details, and is easy to use. This exercise provides code examples for both libraries. If you're a first-time Amazon SageMaker user, we recommend that you use the Amazon SageMaker Python SDK. For more information, see https://sagemaker.readthedocs.io/en/stable/overview.html.

If you're new to Amazon SageMaker, we recommend that you read How Amazon SageMaker Works (p. 2) before starting this exercise.

Topics

- Step 1: Create an Amazon S3 Bucket (p. 17)
- Step 2: Create an Amazon SageMaker Notebook Instance (p. 17)
- Step 3: Create a Jupyter Notebook (p. 18)
- Step 4: Download, Explore, and Transform the Training Data (p. 19)
- Step 5: Train a Model (p. 21)
- Step 6: Deploy the Model to Amazon SageMaker (p. 26)
- Step 7: Validate the Model (p. 30)

- Step 8: Clean Up (p. 35)
- Step 9: Integrating Amazon SageMaker Endpoints into Internet-facing Applications (p. 35)

Step 1: Create an Amazon S3 Bucket

Training a model produces the following

- The model training data
- Model artifacts, which Amazon SageMaker generates during model training

You save these in an Amazon Simple Storage Service (Amazon S3) bucket: You can store datasets that you use as your training data and model artifacts that are the output of a training job in a single bucket or in two separate buckets. For this exercise and others in this guide, one bucket is sufficient. If you already have S3 buckets, you can use them, or you can create new ones.

To create a bucket, follow the instructions in Create a Bucket in the *Amazon Simple Storage Service Console User Guide*. Include `sagemaker` in the bucket name. For example, `sagemaker-datetime`.

> **Note**
> Amazon SageMaker needs permission to access these buckets. You grant permission with an IAM role, which you create in the next step when you create an Amazon SageMaker notebook instance. This IAM role automatically gets permissions to access any bucket that has `sagemaker` in the name. It gets these permissions through the `AmazonSageMakerFullAccess` policy, which Amazon SageMaker attaches to the role. If you add a policy to the role that grants the SageMaker service principal `S3FullAccess` permission, the name of the bucket does not need to contain **`sagemaker`**.

Next Step

Step 2: Create an Amazon SageMaker Notebook Instance (p. 17)

Step 2: Create an Amazon SageMaker Notebook Instance

An Amazon SageMaker notebook instance is a fully managed machine learning (ML) Amazon Elastic Compute Cloud (Amazon EC2) compute instance that runs the Jupyter Notebook App. You use the notebook instance to create and manage Jupyter notebooks that you can use to prepare and process data and to train and deploy machine learning models. For more information, see Explore and Preprocess Data (p. 4).

> **Note**
> If necessary, you can change the notebook instance settings, including the ML compute instance type, later.

To create an Amazon SageMaker notebook instance

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
2. Choose **Notebook instances**, then choose **Create notebook instance**.
3. On the **Create notebook instance** page, provide the following information (if a field is not mentioned, leave the default values):

a. For **Notebook instance name**, type a name for your notebook instance.
b. For **Instance type**, choose `ml.t2.medium`. This is the least expensive instance type that notebook instances support, and it suffices for this exercise.
c. For **IAM role**, choose **Create a new role**, then choose **Create role**.
d. Choose **Create notebook instance**.

   In a few minutes, Amazon SageMaker launches an ML compute instance—in this case, a notebook instance—and attaches an ML storage volume to it. The notebook instance has a preconfigured Jupyter notebook server and a set of Anaconda libraries.

Next Step

Step 3: Create a Jupyter Notebook (p. 18).

Step 3: Create a Jupyter Notebook

Create a Jupyter notebook in the notebook instance you created in Step 2: Create an Amazon SageMaker Notebook Instance (p. 17), and create a cell that gets the IAM role that your notebook needs to run Amazon SageMaker APIs and specifies the name of the Amazon S3 bucket that you will use to store the datasets that you use for your training data and the model artifacts that a Amazon SageMaker training job outputs.

To create a Jupyter notebook

1. Open the notebook instance.
   a. Sign in to the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
   b. Open the notebook instance, by choosing either **Open Jupyter** for classic Juypter view or **Open JupyterLab** for JupyterLab view next to the name of the notebook instance. The Jupyter notebook server page appears:
2. Create a notebook.
   a. If you opened the notebook in Jupyter classic view, on the **Files** tab, choose **New**, and **conda_python3**. This preinstalled environment includes the default Anaconda installation and Python 3.
   b. If you opened the notebook in JupyterLab view, on the **File** menu, choose **New**, and then choose **Notebook.**. For **Select Kernel**, choose **conda_python3**. This preinstalled environment includes the default Anaconda installation and Python 3.
3. In the Jupyter notebook, choose **File** and **Save as**, and name the notebook.
4. Copy the following Python code and paste it into the first cell in your notebook. Add the name of the S3 bucket that you created in Set Up Amazon SageMaker (p. 14), and run the code. The `get_execution_role` function retrieves the IAM role you created when you created your notebook instance.

```
import os
import boto3
import re
import copy
import time
from time import gmtime, strftime
from sagemaker import get_execution_role
```

```
role = get_execution_role()

region = boto3.Session().region_name

bucket='bucket-name' # Replace with your s3 bucket name
prefix = 'sagemaker/xgboost-mnist' # Used as part of the path in the bucket where you
 store data
bucket_path = 'https://s3-{}.amazonaws.com/{}'.format(region,bucket) # The URL to
 access the bucket
```

Next Step

Step 4: Download, Explore, and Transform the Training Data (p. 19)

Step 4: Download, Explore, and Transform the Training Data

Download the MNIST dataset to your notebook instance, review the data, transform it, and upload it to your S3 bucket.

You transform the data by changing its format from `numpy.array` to comma-separated values (CSV). The XGBoost Algorithm (p. 255) expects input in either the LIBSVM or CSV format. LIBSVM is an open source machine learning library. In this exercise , you use CSV format because it's simpler.

Topics

- Step 4.1: Download the MNIST Dataset (p. 19)
- Step 4.2: Explore the Training Dataset (p. 20)
- Step 4.3: Transform the Training Dataset and Upload It to Amazon S3 (p. 21)

Step 4.1: Download the MNIST Dataset

To download the MNIST dataset, copy and paste the following code into the notebook and run it:.

```
%%time
import pickle, gzip, urllib.request, json
import numpy as np

# Load the dataset
urllib.request.urlretrieve("http://deeplearning.net/data/mnist/mnist.pkl.gz",
 "mnist.pkl.gz")
with gzip.open('mnist.pkl.gz', 'rb') as f:
    train_set, valid_set, test_set = pickle.load(f, encoding='latin1')
print(train_set[0].shape)
```

The code does the following:

1. Downloads the MNIST dataset (`mnist.pkl.gz`) from the MNIST Database website to your notebook.
2. Unzips the file and reads the following datasets into the notebook's memory:
   - `train_set` – You use these images of handwritten numbers to train a model.
   - `valid_set` – The XGBoost Algorithm (p. 255) uses these images to evaluate the progress of the model during training.

- `test_set` – You use this set to get inferences to test the deployed model.

Next Step

Step 4.2: Explore the Training Dataset (p. 20)

Step 4.2: Explore the Training Dataset

Typically, you explore training data to determine what you need to clean up and which transformations to apply to improve model training. For this exercise, you don't need to clean up the MNIST dataset.

To explore the dataset

- Type the following code in a cell in your notebook and run the cell to display the first 10 images in `train_set`:.

```
%matplotlib inline
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (2,10)

for i in range(0, 10):
    img = train_set[0][i]
    label = train_set[1][i]
    img_reshape = img.reshape((28,28))
    imgplot = plt.imshow(img_reshape, cmap='gray')
    print('This is a {}'.format(label))
    plt.show()
```

`train_set` contains the following structures:

- `train_set[0]` – Contains images.
- `train_set[1]` – Contains labels.

The code uses the `matplotlib` library to get and display the first 10 images from the training dataset.

Next Step

Step 4.3: Transform the Training Dataset and Upload It to Amazon S3 (p. 21)

Step 4.3: Transform the Training Dataset and Upload It to Amazon S3

The XGBoost Algorithm (p. 255) expects comma-separated values (CSV) for its training input. The format of the training dataset is numpy.array. Transform the dataset from numpy.array format to the CSV format. Then upload it to the Amazon S3 bucket that you created in Step 1: Create an Amazon S3 Bucket (p. 17)

To convert the dataset to CSV format and upload it

- Type the following code into a cell in your notebook and then run the cell.

```
%%time

import struct
import io
import csv
import boto3

def convert_data():
    data_partitions = [('train', train_set), ('validation', valid_set), ('test',
 test_set)]
    for data_partition_name, data_partition in data_partitions:
        print('{}: {} {}'.format(data_partition_name, data_partition[0].shape,
 data_partition[1].shape))
        labels = [t.tolist() for t in data_partition[1]]
        features = [t.tolist() for t in data_partition[0]]

        if data_partition_name != 'test':
            examples = np.insert(features, 0, labels, axis=1)
        else:
            examples = features
        #print(examples[50000,:])


        np.savetxt('data.csv', examples, delimiter=',')



        key = "{}/{}/examples".format(prefix,data_partition_name)
        url = 's3://{}/{}'.format(bucket, key)

 boto3.Session().resource('s3').Bucket(bucket).Object(key).upload_file('data.csv')
        print('Done writing to {}'.format(url))

convert_data()
```

After it converts the dataset to the CSV format, ,the code uploads the CSV file to the S3 bucket.

Next Step

Step 5: Train a Model (p. 21)

Step 5: Train a Model

To train, deploy, and validate a model in Amazon SageMaker, you can use either the Amazon SageMaker Python SDK or the AWS SDK for Python (Boto 3). (You can also use the console, but for this exercise,

you will use the notebook instance and one of the SDKs.) This exercise provides code examples for each library.

The Amazon SageMaker Python SDK abstracts several implementation details, and is easy to use. If you're a first-time Amazon SageMaker user, we recommend that you use it to train, deploy, and validate the model. For more information, see https://sagemaker.readthedocs.io/en/stable/overview.html.

Topics

- Choose the Training Algorithm (p. 22)
- Create and Run a Training Job (Amazon SageMaker Python SDK) (p. 22)
- Create and Run a Training Job (AWS SDK for Python (Boto 3)) (p. 23)

Choose the Training Algorithm

To choose the right algorithm for your model, you typically follow an evaluation process. For this exercise, you use the XGBoost Algorithm (p. 255) provided by Amazon SageMaker, so no evaluation is required. For information about choosing algorithms, see Use Amazon SageMaker Built-in Algorithms (p. 56).

Create and Run a Training Job (Amazon SageMaker Python SDK)

The Amazon SageMaker Python SDK includes the `sagemaker.estimator.Estimator` estimator. You can use this class, in the `sagemaker.estimator` module, with any algorithm. For more information, see https://sagemaker.readthedocs.io/en/stable/estimators.html#sagemaker.estimator.Estimator.

To run a model training job (Amazon SageMaker Python SDK)

1. Import the Amazon SageMaker Python SDK and get the XGBoost container.

```
import sagemaker

from sagemaker.amazon.amazon_estimator import get_image_uri

container = get_image_uri(boto3.Session().region_name, 'xgboost')
```

2. Download the training and validation data from the Amazon S3 location where you uploaded it in Step 4.3: Transform the Training Dataset and Upload It to Amazon S3 (p. 21), and set the location where you store the training output.

```
train_data = 's3://{}/{}/{}'.format(bucket, prefix, 'train')

validation_data = 's3://{}/{}/{}'.format(bucket, prefix, 'validation')

s3_output_location = 's3://{}/{}/{}'.format(bucket, prefix, 'xgboost_model_sdk')
print(train_data)
```

3. Create an instance of the `sagemaker.estimator.Estimator` class.

```
xgb_model = sagemaker.estimator.Estimator(container,
                                          role,
                                          train_instance_count=1,
                                          train_instance_type='ml.m4.xlarge',
                                          train_volume_size = 5,
                                          output_path=s3_output_location,
                                          sagemaker_session=sagemaker.Session())
```

In the constructor, you specify the following parameters:

- `role` – The AWS Identity and Access Management (IAM) role that Amazon SageMaker can assume to perform tasks on your behalf (for example, reading training results, called model artifacts, from the S3 bucket and writing training results to Amazon S3). This is the role that you got in Step 3: Create a Jupyter Notebook (p. 18).
- `train_instance_count` and `train_instance_type` – The type and number of ML compute instances to use for model training. For this exercise, you use only a single training instance.
- `train_volume_size` – The size, in GB, of the Amazon Elastic Block Store (Amazon EBS) storage volume to attach to the training instance. This must be large enough to store training data if you use `File` mode (`File` mode is the default).
- `output_path` – The path to the S3 bucket where Amazon SageMaker stores the training results.
- `sagemaker_session` – The session object that manages interactions with Amazon SageMaker APIs and any other AWS service that the training job uses.

4. Set the hyperparameter values for the XGBoost training job by calling the `set_hyperparameters` method of the estimator. For a description of XGBoost hyperparameters, see XGBoost Hyperparameters (p. 258).

```
xgb_model.set_hyperparameters(max_depth = 5,
                              eta = .2,
                              gamma = 4,
                              min_child_weight = 6,
                              silent = 0,
                              objective = "multi:softmax",
                              num_class = 10,
                              num_round = 10)
```

5. Create the training channels to use for the training job. For this example, we use both `train` and `validation` channels.

```
train_channel = sagemaker.session.s3_input(train_data, content_type='text/csv')
valid_channel = sagemaker.session.s3_input(validation_data, content_type='text/csv')

data_channels = {'train': train_channel, 'validation': valid_channel}
```

6. To start model training, call the estimator's `fit` method.

```
xgb_model.fit(inputs=data_channels,  logs=True)
```

This is a synchronous operation. The method displays progress logs and waits until training completes before returning. For more information about model training, see Train a Model with Amazon SageMaker (p. 4).

Model training for this exercise can take up to 15 minutes.

Next Step

Step 6: Deploy the Model to Amazon SageMaker (p. 26)

Create and Run a Training Job (AWS SDK for Python (Boto 3))

To train a model, Amazon SageMaker uses the CreateTrainingJob (p. 667) API. The AWS SDK for Python (Boto 3) provides the corresponding `create_training_job` method.

When using this method, you provide the following information:

- The training algorithm – Specify the registry path of the Docker image that contains the training code. For the registry paths for the algorithms provided by Amazon SageMaker, see Common Parameters for Built-In Algorithms (p. 58).
- Algorithm-specific hyperparameters – Specify algorithm-specific hyperparameters to influence the final quality of the model. For information, see XGBoost Hyperparameters (p. 258).
- The input and output configuration – Provide the S3 bucket where training data is stored and where Amazon SageMaker saves the results of model training (the model artifacts).

To run a model training job (AWS SDK for Python (Boto 3))

1. Import the `get_image_url` utility function Amazon SageMaker Python SDK and get the location of the XGBoost container.

```
import sagemaker

from sagemaker.amazon.amazon_estimator import get_image_uri

container = get_image_uri(boto3.Session().region_name, 'xgboost')
```

2. Set up the training information for the job. You pass this information when you call `create_training_job`. For more information about the information that you need to send to a training job, see the section called "CreateTrainingJob" (p. 667).

```
#Ensure that the train and validation data folders generated above are reflected in the
 "InputDataConfig" parameter below.
common_training_params = \
{
    "AlgorithmSpecification": {
        "TrainingImage": container,
        "TrainingInputMode": "File"
    },
    "RoleArn": role,
    "OutputDataConfig": {
        "S3OutputPath": bucket_path + "/"+ prefix + "/xgboost"
    },
    "ResourceConfig": {
        "InstanceCount": 1,
        "InstanceType": "ml.m4.xlarge",
        "VolumeSizeInGB": 5
    },
    "HyperParameters": {
        "max_depth":"5",
        "eta":"0.2",
        "gamma":"4",
        "min_child_weight":"6",
        "silent":"0",
        "objective": "multi:softmax",
        "num_class": "10",
        "num_round": "10"
    },
    "StoppingCondition": {
        "MaxRuntimeInSeconds": 86400
    },
    "InputDataConfig": [
        {
            "ChannelName": "train",
            "DataSource": {
                "S3DataSource": {
                    "S3DataType": "S3Prefix",
```

```
                    "S3Uri": bucket_path + "/"+ prefix+ '/train/',
                    "S3DataDistributionType": "FullyReplicated"
                }
            },
            "ContentType": "text/csv",
            "CompressionType": "None"
        },
        {
            "ChannelName": "validation",
            "DataSource": {
                "S3DataSource": {
                    "S3DataType": "S3Prefix",
                    "S3Uri": bucket_path + "/"+ prefix+ '/validation/',
                    "S3DataDistributionType": "FullyReplicated"
                }
            },
            "ContentType": "text/csv",
            "CompressionType": "None"
        }
    ]
}
```

3. Name your training job, and finish configuring the parameters that you send to it.

```
#training job params
training_job_name = 'xgboost-mnist' + strftime("%Y-%m-%d-%H-%M-%S", gmtime())
print("Job name is:", training_job_name)

training_job_params = copy.deepcopy(common_training_params)
training_job_params['TrainingJobName'] = training_job_name
training_job_params['ResourceConfig']['InstanceCount'] = 1
```

4. Call `create_training_job` to start the training job, and wait for it to complete. If the training job fails, print the reason that it failed.

```
%%time

region = boto3.Session().region_name
sm = boto3.Session().client('sagemaker')

sm.create_training_job(**training_job_params)


status = sm.describe_training_job(TrainingJobName=training_job_name)
['TrainingJobStatus']
print(status)
sm.get_waiter('training_job_completed_or_stopped').wait(TrainingJobName=training_job_name)
status = sm.describe_training_job(TrainingJobName=training_job_name)
['TrainingJobStatus']
print("Training job ended with status: " + status)
if status == 'Failed':
    message = sm.describe_training_job(TrainingJobName=training_job_name)
['FailureReason']
    print('Training failed with the following error: {}'.format(message))
    raise Exception('Training job failed')
```

You now have a trained model. Amazon SageMaker stores the resulting artifacts in your S3 bucket.

Next Step

Step 6: Deploy the Model to Amazon SageMaker (p. 26)

Step 6: Deploy the Model to Amazon SageMaker

To get predictions, deploy your model. The method you use depends on how you want to generate inferences:

- To get one inference at a time in real time, set up a persistent endpoint using Amazon SageMaker hosting services.
- To get inferences for an entire dataset, use Amazon SageMaker batch transform.

Topics

- Step 6.1: Deploy the Model to Amazon SageMaker Hosting Services (p. 26)
- Step 6.2: Deploy the Model with Batch Transform (p. 28)

Step 6.1: Deploy the Model to Amazon SageMaker Hosting Services

To deploy a model in Amazon SageMaker, hosting services, you can use either the Amazon SageMaker Python SDK or the AWS SDK for Python (Boto 3). This exercise provides code examples for both libraries.

The Amazon SageMaker Python SDK abstracts several implementation details, and is easy to use. If you're a first-time Amazon SageMaker user, we recommend that you use it. For more information, see https://sagemaker.readthedocs.io/en/stable/overview.html.

Topics

- Deploy the Model to Amazon SageMaker Hosting Services (Amazon SageMaker Python SDK) (p. 26)
- Deploy the Model to Amazon SageMaker Hosting Services (AWS SDK for Python (Boto 3).) (p. 27)

Deploy the Model to Amazon SageMaker Hosting Services (Amazon SageMaker Python SDK)

Deploy the model that you trained in Create and Run a Training Job (Amazon SageMaker Python SDK) (p. 22) by calling the `deploy` method of the `sagemaker.estimator.Estimator` object. This is the same object that you used to train the model. When you call the `deploy` method, specify the number and type of ML instances that you want to use to host the endpoint.

```
xgb_predictor = xgb_model.deploy(initial_instance_count=1,
                                 instance_type='ml.m4.xlarge',
                                 )
```

The `deploy` method creates the deployable model, configures the Amazon SageMaker hosting services endpoint, and launches the endpoint to host the model. For more information, see https://sagemaker.readthedocs.io/en/stable/estimators.html#sagemaker.estimator.Estimator.deploy.

It also returns a `sagemaker.predictor.RealTimePredictor` object, which you can use to get inferences from the model. For information, see https://sagemaker.readthedocs.io/en/stable/predictors.html#sagemaker.predictor.RealTimePredictor.

Next Step

Step 7: Validate the Model (p. 30)

Deploy the Model to Amazon SageMaker Hosting Services (AWS SDK for Python (Boto 3).)

Deploying a model using the AWS SDK for Python (Boto 3) is a three-step process:

1. Create a model in Amazon SageMaker – Send a CreateModel (p. 648) request to provide information such as the location of the S3 bucket that contains your model artifacts and the registry path of the image that contains inference code.
2. Create an endpoint configuration – Send a CreateEndpointConfig (p. 635) request to provide the resource configuration for hosting. This includes the type and number of ML compute instances to launch to deploy the model.
3. Create an endpoint – Send a CreateEndpoint (p. 632) request to create an endpoint. Amazon SageMaker launches the ML compute instances and deploys the model. Amazon SageMaker returns an endpoint. Applications can send requests for inference to this endpoint.

To deploy the model (AWS SDK for Python (Boto 3))

For each of the following steps, paste the code in a cell in the Jupyter notebook you created in Step 3: Create a Jupyter Notebook (p. 18) and run the cell.

1. Create a deployable model by identifying the location of model artifacts and the Docker image that contains the inference code.

```
model_name = training_job_name + '-mod'

info = sm.describe_training_job(TrainingJobName=training_job_name)
model_data = info['ModelArtifacts']['S3ModelArtifacts']
print(model_data)

primary_container = {
    'Image': container,
    'ModelDataUrl': model_data
}

create_model_response = sm.create_model(
    ModelName = model_name,
    ExecutionRoleArn = role,
    PrimaryContainer = primary_container)

print(create_model_response['ModelArn'])
```

2. Create an Amazon SageMaker endpoint configuration by specifying the ML compute instances that you want to deploy your model to.

```
endpoint_config_name = 'DEMO-XGBoostEndpointConfig-' + strftime("%Y-%m-%d-%H-%M-%S",
 gmtime())
print(endpoint_config_name)
create_endpoint_config_response = sm.create_endpoint_config(
    EndpointConfigName = endpoint_config_name,
    ProductionVariants=[{
        'InstanceType':'ml.m4.xlarge',
        'InitialVariantWeight':1,
        'InitialInstanceCount':1,
        'ModelName':model_name,
        'VariantName':'AllTraffic'}])

print("Endpoint Config Arn: " + create_endpoint_config_response['EndpointConfigArn'])
```

3. Create an Amazon SageMaker endpoint.

```
%%time
import time

endpoint_name = 'DEMO-XGBoostEndpoint-' + strftime("%Y-%m-%d-%H-%M-%S", gmtime())
print(endpoint_name)
create_endpoint_response = sm.create_endpoint(
    EndpointName=endpoint_name,
    EndpointConfigName=endpoint_config_name)
print(create_endpoint_response['EndpointArn'])

resp = sm.describe_endpoint(EndpointName=endpoint_name)
status = resp['EndpointStatus']
print("Status: " + status)

while status=='Creating':
    time.sleep(60)
    resp = sm.describe_endpoint(EndpointName=endpoint_name)
    status = resp['EndpointStatus']
    print("Status: " + status)

print("Arn: " + resp['EndpointArn'])
print("Status: " + status)
```

This code continuously calls the `describe_endpoint` command in a `while` loop until the endpoint either fails or is in service, and then prints the status of the endpoint. When the status changes to `InService`, the endpoint is ready to serve inference requests.

Next Step

Step 7: Validate the Model (p. 30)

Step 6.2: Deploy the Model with Batch Transform

To get inference for an entire dataset, use batch transform. Amazon SageMaker stores the results in Amazon S3.

For information about batch transforms, see Get Inferences for an Entire Dataset with Batch Transform (p. 10). For an example that uses batch transform, see the batch transform sample notebook at https://github.com/awslabs/amazon-sagemaker-examples/tree/master/sagemaker_batch_transform/introduction_to_batch_transform.

Topics

- Deploy a Model with Batch Transform (Amazon SageMaker High-level Python Library) (p. 28)
- Deploy a Model with Batch Transform (SDK for Python (Boto 3)) (p. 29)

Deploy a Model with Batch Transform (Amazon SageMaker High-level Python Library)

The following code creates a `sagemaker.transformer.Transformer` object from the model that you trained in Create and Run a Training Job (Amazon SageMaker Python SDK) (p. 22). Then it calls that object's `transform` method to create a transform job. When you create the `sagemaker.transformer.Transformer` object, you specify the number and type of ML instances to use to perform the batch transform job, and the location in Amazon S3 where you want to store the inferences.

Paste the following code in a cell in the Jupyter notebook you created in Step 3: Create a Jupyter Notebook (p. 18) and run the cell.

```
batch_input =
                 's3://{}/{}/test/examples'.format(bucket, prefix) # The location of the
 test dataset

batch_output = 's3://{}/{}/batch-inference'.format(bucket, prefix) # The location to store
 the
results of the batch transform job

transformer = xgb_model.transformer(instance_count=1, instance_type='ml.m4.xlarge',
 output_path=batch_output)

transformer.transform(data=batch_input, data_type='S3Prefix', content_type='text/csv',
 split_type='Line')

transformer.wait()
```

For more information, see https://sagemaker.readthedocs.io/en/stable/transformer.html.

Next Step

Step 7: Validate the Model (p. 30)

Deploy a Model with Batch Transform (SDK for Python (Boto 3))

To run a batch transform job, call the `create_transform_job`. method using the model that you trained in Create and Run a Training Job (AWS SDK for Python (Boto 3)) (p. 23).

To create a batch transform job (SDK for Python (Boto 3))

For each of the following steps, paste the code in a cell in the Jupyter notebook you created in Step 3: Create a Jupyter Notebook (p. 18) and run the cell.

1. Name the batch transform job and specify where the input data (the test dataset) is stored and where to store the job's output.

```
batch_job_name = 'xgboost-mnist-batch' + strftime("%Y-%m-%d-%H-%M-%S", gmtime())

batch_input = 's3://{}/{}/test/examples'.format(bucket, prefix)
print(batch_input)

batch_output = 's3://{}/{}/batch-inference'.format(bucket, prefix)
```

2. Configure the parameters that you pass when you call the `create_transform_job` method.

```
request = \
{
    "TransformJobName": batch_job_name,
    "ModelName": model_name,
    "BatchStrategy": "MultiRecord",
    "TransformOutput": {
        "S3OutputPath": batch_output
    },
    "TransformInput": {
        "DataSource": {
            "S3DataSource": {
                "S3DataType": "S3Prefix",
                "S3Uri": batch_input
            }
        },
```

```
        "ContentType": "text/csv",
        "SplitType": "Line",
        "CompressionType": "None"
    },
    "TransformResources": {
            "InstanceType": "ml.m4.xlarge",
            "InstanceCount": 1
    }
}
```

For more information about the parameters, see the section called "CreateTransformJob" (p. 673).

3. Call the `create_transform_job` method, passing in the parameters that you configured in the previous step. Then call the `describe_transform_job` method in a loop until it completes.

   Paste the following code in a cell in the Jupyter notebook you created in Step 3: Create a Jupyter Notebook (p. 18) and run the cell.

```
sm.create_transform_job(**request)

while(True):
    response = sm.describe_transform_job(TransformJobName=batch_job_name)
    status = response['TransformJobStatus']
    if  status == 'Completed':
        print("Transform job ended with status: " + status)
        break
    if status == 'Failed':
        message = response['FailureReason']
        print('Transform failed with the following error: {}'.format(message))
        raise Exception('Transform job failed')
    print("Transform job is still in status: " + status)
    time.sleep(30)
```

Next Step

Step 7: Validate the Model (p. 30)

Step 7: Validate the Model

Now that you have trained and deployed a model in Amazon SageMaker, validate it to ensure that it generates accurate predictions on new data. That is, on data that is different from the data that the model was trained on. For this, use the test dataset that you created in Step 4: Download, Explore, and Transform the Training Data (p. 19).

Topics

- Step 7.1: Validate a Model Deployed to Amazon SageMaker Hosting Services (p. 30)
- Step 7.2: Validate a Model Deployed with Batch Transform (p. 33)

Step 7.1: Validate a Model Deployed to Amazon SageMaker Hosting Services

If you deployed a model to Amazon SageMaker hosting services in Step 6.1: Deploy the Model to Amazon SageMaker Hosting Services (p. 26), you now have an endpoint that you can invoke to get inferences in real time. To validate the model, invoke the endpoint with example images from the test dataset and check whether the inferences you get match the actual labels of the images.

Topics

- Validate a Model Deployed to Amazon SageMaker Hosting Services (Amazon SageMaker Python SDK) (p. 31)
- Validate a Model Deployed to Amazon SageMaker Hosting Services (AWS SDK for Python (Boto 3)) (p. 32)

Validate a Model Deployed to Amazon SageMaker Hosting Services (Amazon SageMaker Python SDK)

To validate the model by using the Amazon SageMaker Python SDK, use the `sagemaker.predictor.RealTimePredictor` object that you created in Deploy the Model to Amazon SageMaker Hosting Services (Amazon SageMaker Python SDK) (p. 26). For information, see https://sagemaker.readthedocs.io/en/stable/predictors.html#sagemaker.predictor.RealTimePredictor.

To validate the model (Amazon SageMaker Python SDK)

1. Download the test data from Amazon S3.

```
s3 = boto3.resource('s3')

test_key = "{}/test/examples".format(prefix)

s3.Bucket(bucket).download_file(test_key, 'test_data')
```

2. Plot the first 10 images from the test dataset with their labels.

```
%matplotlib inline

for i in range (0, 10):
    img = test_set[0][i]
    label = test_set[1][i]
    img_reshape = img.reshape((28,28))
    imgplot = plt.imshow(img_reshape, cmap='gray')
    print('This is a {}'.format(label))
    plt.show()
```

3. To get inferences for the first 10 examples in the test dataset, call the `predict` method of the `sagemaker.predictor.RealTimePredictor` object.

```
with open('test_data', 'r') as f:
    for j in range(0,10):
        single_test = f.readline()
        result = xgb_predictor.predict(single_test)
        print(result)
```

To see if the model is making accurate predictions, check the output from this step against the numbers that you plotted in the previous step.

You have now trained, deployed, and validated your first model in Amazon SageMaker.

Next Step

Step 8: Clean Up (p. 35)

Validate a Model Deployed to Amazon SageMaker Hosting Services (AWS SDK for Python (Boto 3))

To use the AWS SDK for Python (Boto 3) to validate the model, call the `invoke_endpoint` method. This method corresponds to the InvokeEndpoint (p. 853) API provided by the Amazon SageMaker runtime.

To validate the model (AWS SDK for Python (Boto 3))

1. Download the test data from Amazon S3.

```
s3 = boto3.resource('s3')

test_key = "{}/test/examples".format(prefix)

s3.Bucket(bucket).download_file(test_key, 'test_data')
```

2. Plot the first 10 images from the test dataset with their labels.

```
%matplotlib inline

for i in range (0, 10):
    img = test_set[0][i]
    label = test_set[1][i]
    img_reshape = img.reshape((28,28))
    imgplot = plt.imshow(img_reshape, cmap='gray')
    print('This is a {}'.format(label))
    plt.show()
```

3. Get the Amazon SageMaker runtime client, which provides the `invoke_endpoint` method.

```
runtime_client = boto3.client('runtime.sagemaker')
```

4. Get inferences from the first 10 examples in the test dataset by calling `invoke_endpoint`.

```
with open('test_data', 'r') as f:

    for i in range(0,10):
        single_test = f.readline()
        response = runtime_client.invoke_endpoint(EndpointName = endpoint_name,
                                                  ContentType = 'text/csv',
                                                  Body = single_test)
        result = response['Body'].read().decode('ascii')
        print('Predicted label is {}.'.format(result))
```

5. To see if the model is making accurate predictions, check the output from this step against the numbers you plotted in the previous step.

You have now trained, deployed, and validated your first model in Amazon SageMaker.

Next Step

Step 8: Clean Up (p. 35)

Step 7.2: Validate a Model Deployed with Batch Transform

You now have a file in Amazon S3 that contains inferences that you got by running a batch transform job in Step 6.2: Deploy the Model with Batch Transform (p. 28). To validate the model, check a subset of the inferences from the file to see whether they match the actual numbers from the test dataset.

To validate the batch transform inferences

1. Download the test data from Amazon S3.

```
s3 = boto3.resource('s3')
```

```
test_key = "{}/test/examples".format(prefix)

s3.Bucket(bucket).download_file(test_key, 'test_data')
```

2. Plot the first 10 images from the test dataset with their labels.

```
%matplotlib inline

for i in range (0, 10):
    img = test_set[0][i]
    label = test_set[1][i]
    img_reshape = img.reshape((28,28))
    imgplot = plt.imshow(img_reshape, cmap='gray')
    print('This is a {}'.format(label))
    plt.show()
```

3. Download the output from the batch transform job from Amazon S3 to a local file.

```
s3.Bucket(bucket).download_file(prefix + '/batch-inference/examples.out',
 'batch_results')
```

4. Get the first 10 results from the batch transform job.

```
with open('batch_results') as f:
    results = f.readlines()
for j in range (0, 10):
    print(results[j])
```

5. To see if the batch transform job made accurate predictions, check the output from this step against the numbers that you plotted from the test data.

You have now trained, deployed, and validated your first model in Amazon SageMaker.

Next Step

Step 8: Clean Up (p. 35)

Step 8: Clean Up

To avoid incurring unnecessary charges, use the AWS Management Console to delete the resources that you created for this exercise.

> **Note**
> If you plan to explore other exercises in this guide, you might want to keep some of these resources, such as your notebook instance, S3 bucket, and IAM role.

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/ and delete the following resources:
   - The endpoint. Deleting the endpoint also deletes the ML compute instance or instances that support it.
   - The endpoint configuration.
   - The model.
   - The notebook instance. Before deleting the notebook instance, stop it.
2. Open the Amazon S3 console at https://console.aws.amazon.com/s3/ and delete the bucket that you created for storing model artifacts and the training dataset.
3. Open the IAM console at https://console.aws.amazon.com/iam/ and delete the IAM role. If you created permission policies, you can delete them, too.
4. Open the Amazon CloudWatch console at https://console.aws.amazon.com/cloudwatch/ and delete all of the log groups that have names starting with `/aws/sagemaker/`.

Step 9: Integrating Amazon SageMaker Endpoints into Internet-facing Applications

In a production environment, you might have an internet-facing application sending requests to the endpoint for inference. The following high-level example shows how to integrate your model endpoint into your application.

1. Create an IAM role that the AWS Lambda service principal can assume. Give the role permissions to call the Amazon SageMaker `InvokeEndpoint` API.
2. Create a Lambda function that calls the Amazon SageMaker `InvokeEndpoint` API.
3. Call the Lambda function from a mobile application. For an example of how to call a Lambda function from a mobile application using Amazon Cognito for credentials, see Tutorial: Using AWS Lambda as Mobile Application Backend.